معهد الدلتا العالي لنظم المعلومات الإدارية والمحاسبية

تقرير في مادة **تطبيقات الإنترنت والمنصات الإلكترونية**

الذكاء الاصطناعي في تعلم تطوير الويب

كيف يساعد الذكاء الاصطناعي التوليدي مطور الويب المبتدئ؟

الموضوع المختار: الذكاء الاصطناعي التوليدي كعامل مساعد للمطورين

فهم أكواد HTML وCSS وJavaScript بدل نسخها دون تحقق

إعداد الطالبة **مريم جمال الشحات عوض الفيومي**

الرقم الجامعي: 2501714
السكشن: 136 — المجموعة: (د)
إشراف: د. مها زغلول

17 أكتوبر 2026

المشكلة: صفحة تعمل... لكن هل نفهمها؟

لماذا نحتاج إلى مساعد لا إلى بديل؟

قد تحصل الطالبة المبتدئة على صفحة ويب جاهزة، ثم تعجز عن تفسير سبب تعطل زر أو عدم تغير لون. المشكلة هنا ليست نقص الكود، بل الفجوة بين امتلاك الحل وفهمه. ويزداد الخطر عندما يبدو شرح الأداة مقنعًا؛ إذ توضح OpenAI أن ChatGPT قد يقدّم معلومات خاطئة أو مراجع مختلقة بنبرة واثقة [1]. لذلك لا تكفي الثقة في الأسلوب للحكم على صحة الإجابة.

الذكاء الاصطناعي التوليدي هو نوع من الأدوات ينتج نصوصًا وأكوادًا استجابة لطلب المستخدم. ويمكن توظيف ChatGPT أو Gemini لشرح مقطع صغير، أو تبسيط مصطلح، أو اقتراح تدريب. لكن دوره في هذا التقرير هو **دعم التفكير والتحقق**، وليس كتابة مشروع كامل بدل الطالبة. والأمثلة التالية تعليمية توضيحية، وليست نتائج اختبار فعلي للأداتين أو مقارنة بين أدائهما.

افصلي بين البنية والشكل والسلوك

تحدد HTML عناصر الصفحة ومعانيها؛ وتضبط CSS مظهرها؛ بينما تضيف JavaScript التفاعل. إذا ظهر الزر ولم يستجب، فلا نغيّر الألوان عشوائيًا؛ بل نفحص منطق التفاعل. وإذا لم يتغير لونه، نراجع قواعد التنسيق. هذا الفصل يحول السؤال من «لماذا لا تعمل الصفحة؟» إلى مشكلة صغيرة يمكن فحصها.

مثال HTML: زر واضح المعنى

يوضع هذا المقطع داخل جسم صفحة HTML، وهو أساس الأمثلة اللاحقة:

```
<h1>My first page</h1>
<button id="helloBtn" type="button">Say hello</button>
<p id="message">Welcome</p>
```

يمثل `h1` عنوانًا رئيسيًا، و`p` فقرة. والزر `button` عنصر تفاعلي أصيل يدعم الاستخدام بلوحة المفاتيح؛ أما تحويل `div` إلى زر شكلي فلا يمنحه السلوك نفسه تلقائيًا. وتمنع `type="button"` اعتباره زر إرسال عند وضعه داخل نموذج؛ لكنها لا تكتب فعل النقر نيابة عنا [2]. والمعرّف `id` يتيح الوصول إلى عنصر محدد، ويجب ألا يتكرر في الصفحة [4].

ماذا أطلب؟ «أنا مبتدئة؛ اشرح وظيفة كل عنصر، ولماذا اخترنا button بدل div، ولا تضف مكتبات أو حلًا كاملًا».

كيف أتحقق؟ افتحي الصفحة واضغطي Tab للوصول إلى الزر. لاحظي أن وجوده وحده لا يغيّر الرسالة؛ فالتفاعل سنضيفه لاحقًا. ثم اشرحي الفرق بين ظهور العنصر وتنفيذ وظيفة.

أمثلة تكشف الخطأ بدل إخفائه

مثال CSS: لماذا لم يصبح الزر أحمر؟

ضعي القاعدتين داخل `style` في رأس الصفحة. نفترض هنا عدم وجود قواعد أخرى، وأن القاعدتين في المصدر والطبقة نفسيهما ودون `!important`:

```
#helloBtn { color: navy; }
button { color: red; padding: 10px 16px; }
```

النتيجة المتوقعة: نص الزر كحلي، وليس أحمر؛ لأن محدد المعرّف أعلى خصوصية من محدد نوع العنصر، رغم أن قاعدة `button` جاءت لاحقًا. إذن عبارة «آخر قاعدة تفوز دائمًا» تبسيط خاطئ [3]. أما `padding` فتضيف مساحة داخل حدود الزر، لا خارجه.

ماذا أطلب؟ «فسّر تعارض اللونين، واذكر شروط تفسيرك، ثم أعطني تلميحًا بدل إضافة !important». **كيف أتحقق؟** افحصي الزر في أدوات المطور، وراجعي قاعدة اللون المتجاوزة في لوحة Styles. عطّلي القاعدة الأولى مؤقتًا؛ يفترض أن يظهر الأحمر. بهذا نختبر سبب المشكلة بدل قبول تفسير لفظي فقط.

مثال JavaScript: زر يغيّر رسالة

أضيفي المقطع بعد عناصر المثال الأول وقبل إغلاق `body`، حتى تكون العناصر موجودة عند البحث عنها:

```
<script>
const button = document.getElementById("helloBtn");
const message = document.getElementById("message");
button.addEventListener("click", () => {
  message.textContent = "Hello, Maryam!";
});
</script>
```

يبحث `getElementById` عن العنصر بمعرّفه؛ ثم يسجل `addEventListener` دالة تعمل عند النقر، لا عند مجرد تسجيلها. وتستبدل `textContent` نص الفقرة [4، 5، 6]. النتيجة المتوقعة: تبقى Welcome قبل النقر، وتظهر Hello, Maryam! بعده.

ماذا أطلب؟ «اربط كل اسم بالعنصر المقابل، واشرح وقت تنفيذ الدالة، ثم اسألني عما سيحدث إذا تغير معرّف الزر». **كيف أتحقق؟** غيّري `helloBtn` في HTML فقط إلى `helloButton`؛ سيعيد البحث عن الاسم القديم `null`، وقد يظهر `TypeError` عند محاولة تسجيل الحدث عليه [4]. افتحي Console، واقرئي الخطأ ثم أصلحي تطابق الاسمين.

اختبار الفهم الحقيقي: أغلقي المحادثة، وغيّري نص الرسالة بنفسك، ثم اشرحي لماذا احتجنا HTML وJavaScript معًا. نجاح النسخ ليس دليلًا كافيًا على التعلم.

من الفوائد والمخاطر إلى خطة عمل

الإيجابيات: شرح يناسب المبتدئ

تتيح المساعدة التوليدية طلب الشرح بالعربية، وتقسيم الكود إلى خطوات، وإعادة تفسير المصطلحات، واقتراح أخطاء تدريبية صغيرة. ويمكن أن تقلل وقت التعثر في سؤال بسيط وتساعد الطالبة على مواصلة التعلم. لكن هذه فوائد محتملة تتوقف على جودة الطلب والتحقق؛ فلا يصح افتراض أن كل إجابة دقيقة أو أن كل استخدام يحسن المستوى [1، 7].

السلبيات والمخاطر: أين نقع في الفخ؟

الإجابة المقنعة الخاطئة: قد تفسر الأداة خطأ CSS بأنه مشكلة في المتصفح، فتدفع المبتدئ إلى تعديلات لا تعالج السبب. وقد تضيف كودًا أكثر تعقيدًا من المطلوب أو تستشهد بمرجع لا يدعم كلامها [1].

الاعتماد وضعف التعلم: تكرار طلب الحل الكامل قد يخفي عدم فهم الأساسيات. ويظهر ذلك عندما تعجز الطالبة عن تعديل اسم متغير أو إعادة بناء المثال دون المحادثة. هذا خطر تعليمي يجب منعه بتصميم التدريب، لا بالاعتماد على حسن النية وحده.

الخصوصية والأمان والأمانة: لصق كلمات مرور أو مفاتيح وصول أو بيانات زملاء في المحادثة قد يكشف معلومات لا نملك حق مشاركتها. كذلك لا ينبغي تشغيل أوامر تثبيت أو إرسال بيانات إلى روابط غير مفهومة لمجرد أن المساعد اقترحها. وتشدد إرشادات اليونسكو على حماية البيانات وبقاء الإنسان مسؤولًا عن الاستخدام التعليمي [7].

خمس خطوات تجعل المساعدة تدريبًا

1. **حددي المشكلة:** اذكري مستواك، وما المتوقع وما الذي حدث، وأرفقي أصغر مقطع لازم بعد حذف البيانات الحساسة. لا ترسلي المشروع كله إذا كان الخطأ في سطرين.
2. **اطلبي تفسيرًا وتوقعًا:** «اشرح السبب، وحدد موضع الكود، وما الذي يفترض أن أراه، ولا تعرض الحل قبل تلميح». ثم اكتبي توقعك أنت قبل التشغيل.
3. **راجعي المرجع:** افتحي صفحة التوثيق المرتبطة بالخاصية في MDN، وطابقي الادعاء بالنص، لا بعنوان الرابط فقط. وجود مصدر لا يعني أنه يدعم الإجابة [1].
4. **جرّبي تغييرًا واحدًا:** قارني المتوقع بما يظهر، وافحصي Console وStyles. أصلحي السبب، ثم أعيدي التجربة؛ فاختفاء رسالة الخطأ لا يثبت وحده صحة السلوك أو جودة التصميم.
5. **أعيدي البناء باستقلال:** أغلقي المحادثة ونفذي تعديلًا مشابهًا مع شرح السبب بكلماتك. احتفظي بملاحظة قصيرة: الخطأ، وسببه، وكيف تحققتِ منه؛ والتزمي بتعليمات الدكتورة بشأن الإفصاح عن المساعدة.

مقياس النجاح: ليس عدد الأسطر التي حصلتِ عليها، بل قدرتك على تفسيرها، واكتشاف خطئها، وتعديلها دون الاعتماد الدائم على الأداة.

كيف نحسن الاستخدام مستقبلًا؟

مقترحات تجعل المساعدة أكثر فائدة

أقترح أن تقدم المنصات التعليمية وضعًا للمبتدئ يبدأ بالتلميحات والأسئلة قبل الحل، ويربط الخصائص بمراجع أصلية، ويوضح ما لم يتحقق منه بدل إظهار يقين زائد. ويمكن للمعهد تقديم تدريبات قصيرة تجمع الكود ولقطة النتيجة وتفسير الطالبة للتعديل؛ حتى يقيس التقييم الفهم لا مجرد اكتمال الصفحة. وهذه مقترحات تطوير، لا خصائص مثبتة هنا في ChatGPT أو Gemini.

الخاتمة: نحول السلبيات إلى فرص للتعلم

يمكن جعل خطر الإجابة الخاطئة فرصة لتعلم التحقق من التوثيق، والاعتماد الزائد دافعًا إلى تدريب مستقل بعد كل شرح، وتعقيد الأكواد سببًا لطلب أصغر حل ممكن. وتصبح مخاطر الخصوصية أقل عندما نحذف البيانات الحساسة ونشارك الحد الضروري فقط. هذه الخطوات لا تلغي السلبيات، لكنها تجعلها قابلة للإدارة بدل تركها تؤثر في التعلم [1، 7].

ومن وجهة نظري كطالبة، أفضل مساعدة هي التي تجعلني أستطيع شرح الكود بعد انتهاء المحادثة، لا التي تعطيني أطول إجابة. لذلك أرى أن استخدام الذكاء الاصطناعي مع التوثيق والتجربة قد يساعد المبتدئ، بشرط أن يبقى القرار والفهم مسؤوليتنا. فالهدف ليس صفحة تعمل مرة واحدة، بل متعلم يعرف لماذا تعمل وكيف يصلحها.

المراجع والمصادر

تطابق الأرقام التالية الإحالات داخل النص. الروابط قابلة للنقر، وتوثق المعلومات التقنية والمبادئ العامة، لا نتائج تجربة على الأداتين.

[1] **OpenAI Help Center.** Does ChatGPT tell the truth? — حدود الدقة، والإجابات الواثقة الخاطئة، والتحقق من المراجع.

[2] **MDN Web Docs.** The button element — دلالة الزر، ولوحة المفاتيح، وخاصية النوع.

[3] **MDN Web Docs.** Specificity — أوزان المحددات وشروط المقارنة بين قواعد التنسيق.

[4] **MDN Web Docs.** Document: getElementById() — المعرف الفريد، وحساسية الأحرف، والقيمة null.

[5] **MDN Web Docs.** EventTarget: addEventListener() — ربط الدالة بحدث المستخدم.

[6] **MDN Web Docs.** Node: textContent — قراءة النص واستبداله داخل العنصر.

[7] **Miao, F. & Holmes, W. (2023). UNESCO.** Guidance for generative AI in education and research — الاستخدام المتمحور حول الإنسان، وحماية البيانات، والتصميم التعليمي.